

미니프로젝트 2차

프로젝트 완료 보고서*

AI 수도권 17조

공다연, 김경순, 김연주, 김현민, 노경천, 박종호, 심경민, 장윤재

목차

01

프로젝트 요약

2p

02

강의 제작
설계 방향성

3p

03

State, Graph

4p

04

실행 결과

5p

05

첨부자료

6p

프로젝트 요약

프로젝트 배경

기업 내에서 다양한 형태의 자료가 지속적으로 생산됨
대부분 정적 자료 형태 → 디지털 전환 시대, 교육/ 보고 영상을 자동 제작 Agent 구축

프로젝트 목적

청자 수준을 고려한 강의 발표 PPT 영상 제작

예시

- PPT 내용을 그대로 읽지 않고 자연스러운 강의 스크립트를 생성하여 상대방을 이해
- 한국어뿐만 아니라 다국적 언어로 제공

기능 설명

- 향후 동향 추가(검색 API)
- 자막 · 번역 추가

기대 효과

- 즉시성: 신제품 안내 자료 → 당일 교육 영상 제작 가능
- 효율성: 사내 보고·발표 준비 시간 대폭 단축
- 표준화: 발표 톤·길이를 자동 조절 → 전달 품질 균일화

강의 제작 설계 방향성

문제점 파악을 우선적으로 진행 이후, 문제 해결을 위한 기능과, 편의 기능 추가

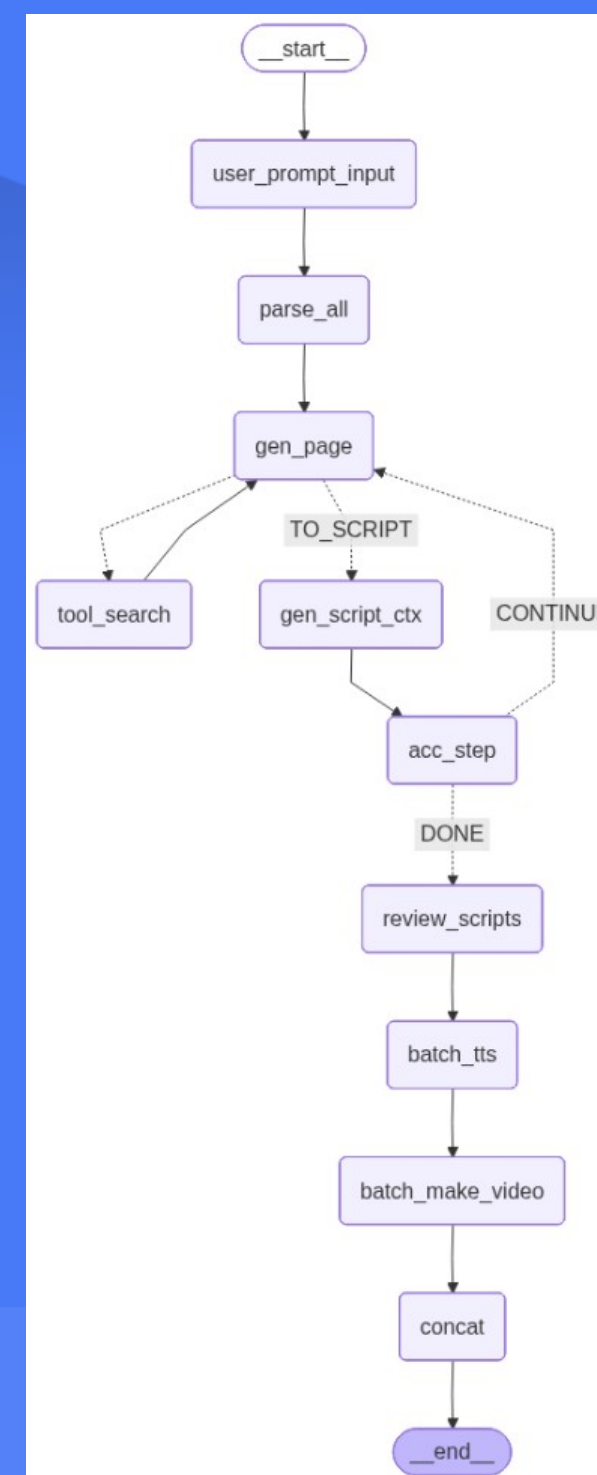
	주요 이슈	해결 방법	함수명
스크립트 퀄리티 부족	PPT를 단순하게 읽기만 하여 강의같지 않음, 설득력 부족	→ 예시를 들어 설명하는 기능, 검색 기능을 통해 내용 보충	gen_script_ctx node_tool_search
슬라이드 별 스크립트 생성 문제점	슬라이드 별 톤, 목소리 불일치 2, 3번 페이지에 불필요한 인사	→ 전체 스크립트 통합 검토 기능, 프롬프트 최적화	review_scripts gen_script_ctx
추가 기능	사용자 타겟 설정 편의성 추가 자막 기능 등	→ 강의 방식 사용자 설정 기능 향후 동향 추가 자막 및 번역 기능	user_prompt_input search_and_summarize build_subtitle_filter, script_to_srt

State, Graph 구조

State

- 기본 환경 변수
- PPT 슬라이드 정보
- 현재 슬라이드 정보
- 전체 결과 누적 리스트
- 최종 결과물(영상 경로)

Graph 구조



결과물

https://drive.google.com/file/d/1XjFf40I26qEudheiEKVI5yxnyOwBQJAY/view?usp=drive_

불만족 고객 44명 중 38명을 놓치고 있었습니다

19,278건의 설문 데이터 분석 결과,
기존 모델은 불만족 고객의 77%만 탐지했습니다.
미세조정 모델 적용 후 탐지율 97%로 개선—
이탈 예측 자동화 시스템 도입을 제안합니다.

6반 17조 | 조장: 심경민 | 2026.04.14 ~ 04.15

19,278건

분석 대상 설문 데이터

Recall 97%

미세조정 후 불만족 탐지율

+19.9%p

Recall 개선폭 (77%→97%)

결과 및 문제점

결과

사용자 타켓을 강의영상 제작으로 한정하였을때 만족스러운 결과
PPT에 작성된 내용이외에도 자연스럽게 강의를 확인할수 있음
이외에도 다국어 번역, 자막 생성 기능등이 원활하게 작동

· 문제점 및 향후발전 계획 ·

- 사용자 타켓이 강의영상이 아닌 사업발표회 등으로 변경 되었을때 자연스럽게지 않음
 - ex)향후 동향이 신기술발표회와 같은 상황에서는 발표 내용에 방해
- PPT의 특성을 뽑아 청자 타켓을 파악하는 기능을 통해 검색기능과 결합하여 다양한 타켓층에 사용 가능하도록 기대
 - 타켓층을 변수를 통해 받아오는 기능을 이용하여 자연스럽게 결합 가능

추가자료

State

```
class State(TypedDict):  
    # 1. 기본 환경 변수  
    pptx_path: str  
    work_dir: str  
    prompt: Dict  
  
    # 2. PPT 정보 및 루프(반복) 제어  
    slides: List[Dict[str, Any]] # 전체 슬라이드의 텍스트/이미지 정보  
    n_slides: int                # 총 슬라이드 수  
    slide_index: int             # 현재 처리 중인 슬라이드 번호 (0부터 시작)  
  
    # 3. 현재 슬라이드 처리용 임시 변수 (루프 돌 때마다 갱신됨)  
    cur_search_context: str  
    cur_page_content: str  
    cur_script: str  
    cur_audio: str  
    cur_video: str  
    cur_search_done: bool  
  
    # 4. 전체 결과 누적 리스트 (node_accumulate_and_step에서 append됨)  
    search_context: List[str]  
    page_contents: List[str]  
    scripts: List[str]  
    audios: List[str]  
    videos: List[str]  
  
    # 5. 최종 결과물  
    final_scripts: List[str]  
    final_video: str
```


추가자료

강의방식 사용자 설정

```

# —— 설문 항목 정의 (설명, 기본값, LLM추론가능여부) ——
PROMPT_SCHEMA = {
    # ① 강의 기본 정보
    "subject": ("강의 주제 / 과목명", None, False),
    "target_audience": ("대상 청중 수준 (예시: 입문자, 중급자, 전문가 등)", "입문자", False),
    "lang": ("강의 음성 · 자막 언어 (예시: 한국어, 영어 등)", "한국어", False),

    # ② 말투 / 음성
    "tone": ("강의 말투", "친절하고 명료한 강의 톤", False),
    "voice": ("TTS 음성\n\n"
              "• alloy\n"
              "• ash\n"
              "• ballad\n"
              "• coral\n"
              "• echo\n"
              "• fable\n"
              "• nova\n"
              "• onyx\n"
              "• sage\n"
              "• shimmer\n"
              "• verse\n"
              "• marin\n"
              "• cedar\n", "alloy", False),

    # ③ 콘텐츠 스타일
    "style": ("강의 스타일", "슬라이드별 예시와 핵심 설명 중심", False),
    "depth": ("설명 깊이 (예시: 개요, 표준, 심화)", "표준", False),
    "example_type": ("선호 예시 유형 (예시: 실제사례, 비유, 코드, 수식)", "실제 사례", False),

    # ④ 스크립트
    "slide_duration": ("슬라이드당 목표 시간(초) (60초 이상 권장)", "60", False),

    # ⑤ 청중 맞춤
    "prior_knowledge": ("청중 사전 지식 (예시: 없음, 기초, 실무경험 등)", "기초 개념 이해", False),
    "interest_focus": ("청중 주요 관심사 (예시: 취업, 학문, 자격증 등)", "취업 · 실무", False),
}

```

```

def node_user_prompt_input(state: State) -> State:
    if not state.get("pptx_path"):
        answer = input("PPTX 파일 경로를 입력하세요 (예: /content/lecture.pptx):")
        state["pptx_path"] = answer.strip()

    prompt = state.get("prompt", {})
    slides = state.get("slides", [])
    print("다음 강의 설정 내용을 입력하세요")
    for field in list(PROMPT_SCHEMA.keys()):
        desc, default, inferrable = PROMPT_SCHEMA[field]
        answer = input(
            f"{desc}\n\n"
            f"(기본값: {default} / Enter 시 기본값 적용):"
        )
        prompt[field] = answer.strip() if answer.strip() else default

    state["prompt"] = prompt
    return state

```

추가자료

스크립트 생성 프롬프트

```
# 2. 메시지 구성 ----- # [슬라이드 핵심 요약본]을 뼈대로 70%, [슬라이드 외부 추가본]을 30% 활용해서: 만약 너무 외부만 사용한다면
## 시스템 메시지(SystemMessage)
system_msg = f'''
    [언어 지침 - 최우선 적용] {lang_inst}

    당신은 1타 강사처럼 자연스럽게 귀에 쏙쏙 들어오는 발표 대본을 작성하는 전문 에이전트입니다.
    [이전 슬라이드 스크립트]와 맥락과 흐름이 이어지도록 이전 내용은 반복하지 말고 자연스럽게 연결만 하세요.
    제공된 [슬라이드 핵심 요약본]을 활용해서, [발표 목소리]로 설명하듯 말하는 350~450자 글자 분량의 강의 핵심 요약 스크립트를 작성하세요.

    [페이지별 조건]
    1. 첫 번째 페이지인 경우 ({slide_index}가 0일 때):
        - 활기찬 환영 인사와 함께 전체 주제를 소개하며 시작하세요.
    2. 중간 페이지인 경우 ({slide_index}>1일 때):
        - **인사 없이** [이전 슬라이드 스크립트]의 내용을 아주 짧게 언급하며 자연스럽게 본론으로 연결하세요.
    3. 마지막 페이지 마무리인 경우 ({slide_index}가 {n_slides}일 때):
        -마지막에는 결론으로 유도하는 맺음말로 마무리하세요.

    [스크립트 구성 전략]
    - 슬라이드 텍스트를 그대로 읽지 마세요. 슬라이드는 시각 자료일 뿐입니다.
    - "이 화면을 보시면", "여기 그래프의 이 부분을 주목해 주세요" 처럼 실제 발표자가 화면을 가리키며 설명하는 듯한 '지시어'를 섞으세요.
    - 핵심 요약본({cur_page_content})의 단어 사이사이에 "왜냐하면", "즉", "예를 들어" 같은 연결어를 넣어 논리적으로 설명하세요.

    [적용할 말투 및 스타일]
    - 톤(Tone): [발표 톤]
    - 스타일(Style): [발표 스타일]

    [절대 금지 규칙 - TTS 음성 변환용이므로 엄격히 지킬 것]
    - **페이지를 넘어갈 때, 이전 스크립트와 "톤, 스타일, [발표 목소리]"을 같도록 유지시킬 것**
    - 마크다운 특수기호(*, #, -, ~ 등) 절대 사용 금지
    - 괄호를 사용한 행동 지문(예: (웃으며), (화면을 가리키며) 등) 절대 사용 금지
    - 볼릿 포인트 사용 금지
    - 오로지 아나운서가 프롬프트를 보고 읽을 수 있는 '순수 텍스트(구어체)'만 출력하세요.

    [출력 형식]
    - 위에서 지정한 언어로만 출력하세요. 마크다운, 괄호 주석, 이모지 금지.
    ...
# 중간에 인사, 톤, 스타일 변경됨, 1 페이지 너무 짧게 나옴
```


추가자료

Gradio

```
import os, time, shutil
import gradio as gr

VOICES = ["alloy", "aria", "verse", "shimmer", "coral", "sage", "nova", "amber"]

def run_pipeline_ui(pptx_file, tone, voice):

    # 작업 디렉터리 & 업로드 저장
    work_dir = os.path.join("./webio", f"run-{int(time.time())}")
    os.makedirs(work_dir, exist_ok=True)
    pptx_path = os.path.join(work_dir, "input.pptx")
    shutil.copy(pptx_file.name, pptx_path)

    # 초기 state (State 스키마에 맞춤)
    state = {
        "pptx_path": pptx_path,
        "work_dir": work_dir,
        "prompt": {
            "voice": voice or "alloy",
            "tone": tone or "친절하고 명료한 강의 톤",
            "style": "예시와 핵심 요점 중심",
        },
    }

    # 그래프 실행
    final_state = app.invoke(state)
    video_path = final_state.get("final_video")

    return video_path, video_path

# ----- Gradio UI -----
with gr.Blocks(title="AI 강사 Agent") as demo:
    gr.Markdown("### PPT --> 강의영상 자동 제작")

    with gr.Row():
        inp_ppt = gr.File(label="PPTX 업로드", file_types=[".pptx"])
        with gr.Column():
            inp_tone = gr.Textbox(value="친절하고 명료한 강의 톤", label="강의 작성 톤 (프롬프트)")
            inp_voice = gr.Dropdown(VOICES, value="alloy", label="TTS Voice")

    run_btn = gr.Button("실행", variant="primary")

    out_video = gr.Video(label="최종 동영상 미리보기", interactive=False)
    out_download = gr.DownloadButton(label="동영상 다운로드")

    run_btn.click(run_pipeline_ui, [inp_ppt, inp_tone, inp_voice], [out_video, out_download])

demo.launch()
```